

DataOps Mastery: Advanced CI/CD and Pipeline Automation

Module 3 – Bundle anatomy and your first deployment

Asst. Prof. Dr. Santitham Prom-on

Department of Computer Engineering, Faculty of Engineering
King Mongkut's University of Technology Thonburi

Course Overview

DAY 1

FOUNDATIONS OF GIT, YAML AND DATABRICKS



MODULE 1

Git that survives a real team

- CI/CD concepts: why notebook-driven delivery breaks
- Introduction to Git, version control in data work
- add, commit, push, pull
- Branching, merging, conflicts
- Databricks Repos: Git inside the workspace



MODULE 2

YAML and the Databricks workspace

- YAML, deep-dive- syntax, context, mappings, sequences
- The traps that break CI; anchors and reuse
- Databricks workspace (re)overview
- Cluster, serverless and job compute
- Databricks CLI commands, operations and authentication



DAY 2

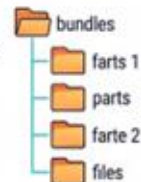
DATABRICKS ASSET BUNDLE



MODULE 3

Bundle anatomy and your first deployment

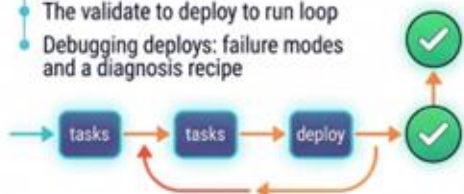
- What problem do bundles solve?
- Anatomy of databricks.yml
- Variables, targets and development mode
- Initialize and explore a bundle



MODULE 4

Build, deploy, run, iterate

- Defining jobs, tasks and clusters in YAML
- Multi-task pipelines and dependencies
- Targets and variable overrides
- The validate to deploy to run loop
- Debugging deploys: failure modes and a diagnosis recipe



DAY 3

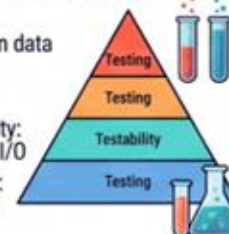
TESTING AND CONTINUOUS INTEGRATION



MODULE 5

Testing pipelines: writing code you can verify

- What's worth testing in data and ML work
- The testing pyramid
- Designing for testability: separating logic from I/O
- Writing the test cases: pytest, fixtures, Spark assertions



MODULE 5

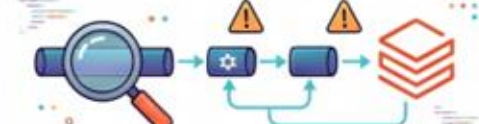
CI/CD principles: workflows, jobs, steps

- GitHub Actions architecture and composition
- Workflows, jobs, steps, runners and triggers
- Marketplace actions
- Securing workflows, secrets and masking



DAY 4

E2D-TO-END CI/CD ON DATABRICKS



MODULE 6

Wiring GitHub Actions to Databricks

- Authentication: service principals and OAuth
- Environments, environment secrets and approval gates
- Workflow pattern: validate on PR, deploy on merge
- Multi-environment promotion; reusable workflows in practice



MODULE 8

Production-readiness and capstone

- Integration and smoke tests against a deployed bundle
- Monitoring and alerting
- Rollback and troubleshooting patterns
- Capstone



Module Overview

- State what a bundle contains and what a deployment of it writes to the workspace.
- Read databricks.yml and name the function of each top-level key.
- Predict the configuration the CLI resolves for a target, and confirm it.
- Generate a project with bundle init and account for every file it produces.
- Validate, plan, deploy, run, destroy and redeploy from the command line.
- Identify which copy of a notebook a deployed job is executing.

What a bundle is

Part 1

Terminology and versions

- Databricks Asset Bundles were renamed Declarative Automation Bundles in March 2026
- The CLI is at version 1.x
- New bundles use the direct deployment engine

Where workspace state comes from

The job you created in Module 2, considered as an object someone else has to maintain

- The notebook arrived through workspace import, from a path on your laptop.
- The job was created by an API call whose body you assembled on the command line.
- The compute was chosen when the job was created, by cluster id.
- Recreating the job in a second workspace requires all three, and the workspace records none of them.
- Reviewing a change to the job before it happens is not possible, because there is nothing to read.

This is the state Module 2 deliberately left you in.

What automated deployment requires

Three properties, and the mechanism that supplies each

Requirement	What it means	Supplied by
Reproducibility	The same input produces the same workspace state	A declaration held in a file
Environment parity	Development and production differ only where declared	One file, several targets
Reviewability	A change is legible before it is applied	A text file, and a plan command

A bundle is the unit that provides all three.

A bundle is a source tree with a declaration

The directory is the unit that is deployed

ON DISK

```
my-project/  
  databricks.yml      #the declaration  
  pyproject.toml  
  resources/  
    sample_job.job.yml  
  src/  
    sample_notebook.ipynb  
    my_project/main.py  
  tests/
```

WHAT databricks.yml DECLARES

```
bundle:      the identity of this project  
include:     which other files to merge  
variables:   values that differ by target  
targets:     where it can be deployed  
resources:   what the workspace should  
              contain
```

Part B takes these five keys in order.

What a deployment consists of

Four things are written under one root path, derived from the bundle name and the target

```
/Workspace/Users/<you>/.bundle/<bundle name>/<target>/
```

<code>files/</code>	the synchronised source tree
<code>artifacts/</code>	anything built during deployment, such as a wheel
<code>resources/</code>	the definitions the workspace was given
<code>state/</code>	what the CLI created, recorded so it can compute a difference

and, outside the root path, the jobs and pipelines themselves.

The job appears under Workflows like any other. The four directories above are what makes it reproducible.

The command surface

databricks bundle, with the six used M3-M4 marked

Command	What it does	Used in
init	Generate a project from a template	M3
validate	Resolve the configuration and check it against the schema	M3
plan	Report the actions a deployment would take	M3
deploy	Build, upload, create or update resources, record state	M3
run	Start a job or pipeline and report its terminal state	M3
destroy	Delete the resources and files this bundle created	M3
summary, open	List deployed resources; open one in a browser	M3
generate	Write configuration from a job that already exists	M4
sync, schema, deployment	One-way file sync; JSON schema; state operations	reference

The lifecycle, against Module 2's sequence

The left column is what you ran yesterday afternoon

Module 2	Module 3
<code>workspace import to upload the notebook</code>	<code>bundle deploy</code>
<code>jobs create with an assembled JSON body</code>	<code>bundle deploy</code>
<code>jobs run-now with the job id</code>	<code>bundle run <key></code>
<code>jobs get-run in a polling loop</code>	<code>bundle run reports the terminal state</code>
<code>jobs get-run-output with the task run id</code>	<code>bundle run prints the run URL</code>
<code>delete by hand, or leave it behind</code>	<code>bundle destroy</code>

The anatomy of databricks.yml

Part 2

The smallest bundle that deploys

Three keys. The next fifteen slides add one mechanism each.

```
bundle:
  name: m3demo

targets:
  dev:
    default: true
    workspace:
      host: https://adb-1234567890.7.azuredatabricks.net

resources:
  jobs:
    sample_job:
      name: sample_job
      tasks:
        - task_key: notebook_task
          notebook_task:
            notebook_path: ./src/sample_notebook.ipynb
```

bundle: the identity of the project

Two keys, both consequential

- **name**
Identifies the bundle, and appears in every path derived from it. Two bundles with the same name, deployed by the same user, occupy the same root path.
- **uuid**
Generated once, at initialisation, and written into the file. It identifies the bundle to the workspace independently of its name, so the name can change.

Nothing else belongs under bundle:. Deployment settings belong to a target.

targets: a named destination

A target names a workspace and the settings that apply when deploying there

```
bundle:
  name: m3demo

targets:
  dev:
    default: true
    workspace:
      host: https://adb-1234567890.7.azuredatabricks.net

  prod:
    workspace:
      host: https://adb-1234567890.7.azuredatabricks.net
      root_path: /Workspace/Users/you@example.com/.bundle/${bundle.name}/${bundle.target}
```

dev and prod are names chosen by convention, not keywords. Everything declared outside targets: is common to all of them.

Selecting a target

Which workspace a command acts on

- **databricks bundle deploy -t prod**

The -t, or --target, flag selects one target by name. Every bundle command accepts it.

- **default: true**

Marks the target used when the flag is omitted. The generated template puts it on dev.

- **Nothing prevents prod from being the default**

Which is the argument for naming the target explicitly in any command whose effect you would not want to discover afterwards.

Workspace paths are derived, not written

From the bundle name and the selected target, resolved here for target dev

Key	Resolved value	Holds
<code>root_path</code>	<code>.../Users/<you>/.bundle/m3demo/dev</code>	everything below
<code>file_path</code>	<code><root_path>/files</code>	the synchronised source tree
<code>artifact_path</code>	<code><root_path>/artifacts</code>	built artifacts, such as a wheel
<code>resource_path</code>	<code><root_path>/resources</code>	generated resource definitions
<code>state_path</code>	<code><root_path>/state</code>	the deployment state and metadata

Any of the five may be overridden. The generated prod target overrides `root_path`, for a reason given on the production mode slide.

resources: what the workspace should contain

A mapping from resource type, to resource key, to the definition

- `resources.jobs.<key>` declares a job. `resources.pipelines.<key>` declares a pipeline.
- The type list also covers clusters, model serving endpoints, dashboards, schemas, volumes and apps.
- A resource is declared once and deployed to every target, unless a target overrides it.
- This module deploys the resources the template writes. Module 4 authors them.

A job resource

One task, taken from the generated project and reduced to its required keys

```
resources:
  jobs:
    sample_job:                                # the resource key
      name: sample_job                        # the label shown in the workspace

    tasks:
      - task_key: notebook_task
        notebook_task:
          notebook_path: ../src/sample_notebook.ipynb
```

notebook_path is relative to the file that declares it, so a resource file and the sources it names move together.

Resource key and resource name

Two identifiers on the same object, with different consumers

THE KEY

`sample_job`

Used by the CLI and by references from other resources:

```
databricks bundle run sample_job  
${resources.jobs.sample_job.id}
```

Changing it detaches the resource from its deployment state.

THE NAME

`sample_job`

The label shown under Workflows.

Development mode rewrites it to `[dev santitham_pro] sample_job` and leaves the key untouched.

Changing it affects nothing else.

include: composing the configuration from several files

Every matching file is merged into one configuration before anything else happens

```
# databricks.yml
bundle:
  name: m3demo

include:
  - resources/*.yaml

# resources/sample_job.job.yaml
resources:
  jobs:
    sample_job:
      name: sample_job
      ...
```

The generated project puts one resource in each file, so databricks.yml holds only bundle-level configuration.

variables: declaring a parameter

Declared once at the top level, assigned per target

```
variables:
  catalog:
    description: The catalog to use
  schema:
    description: The schema to use

targets:
  dev:
    variables:
      catalog: training
      schema: ${workspace.current_user.short_name}
  prod:
    variables:
      catalog: training
      schema: prod
```

A variable used but not declared is an error. A variable declared with neither a default nor a target assignment is prompted for.

Where a variable's value comes from

Four sources, in increasing order of precedence

Source	Form	Typical use
The declaration	<code>default: value under variables:</code>	a value that rarely changes
The selected target	<code>variables: block inside the target</code>	the value for this environment
The environment	<code>DATABRICKS_BUNDLE_VAR_catalog=x</code>	a value a server supplies
The command line	<code>--var catalog=x</code>	one deployment that differs

Substitution namespaces

Four appear in the generated file, and they do not all resolve at the same time

Reference	Resolves to	Resolved during
<code>\${bundle.name}</code> , <code>\${bundle.target}</code>	bundle metadata	configuration resolution
<code>\${workspace.file_path}</code>	a derived workspace path	configuration resolution
<code>\${workspace.current_user.short_name}</code>	the authenticated user, normalised	configuration resolution
<code>\${var.catalog}</code>	the variable's value for this target	configuration resolution
<code>\${resources.pipelines.etl.id}</code>	the id of a deployed resource	deployment

The last row is the only one still written as a reference in validate output, because the resource it names does not exist until it is deployed.

mode: development

A target may declare a mode, which applies computed settings on top of the declaration

- The declaration is unchanged. The mode adds presets, which the CLI computes and applies during resolution.
- The purpose is that a development deployment cannot be mistaken for, or collide with, a production one.
- Presets can be overridden individually, and a setting on a resource outranks a preset.

The presets development mode computes

Present in the resolved configuration, absent from every file

Preset	Value	Effect on the deployment
<code>name_prefix</code>	<code>[dev santitham_pro]</code>	every resource name is prefixed
<code>tags</code>	<code>dev: santitham_pro</code>	jobs and pipelines carry the tag
<code>trigger_pause_status</code>	<code>PAUSED</code>	schedules and triggers do not fire
<code>jobs_max_concurrent_runs</code>	<code>4</code>	repeated runs during iteration do not queue
<code>pipelines_development</code>	<code>true</code>	pipelines run in development mode

The deployment lock is also disabled, and `--cluster-id` may override the declared compute. Neither is permitted in production mode.

mode: production

Computes no presets, and enforces constraints instead

- Paths must not be user-specific, which is why the generated prod target writes `root_path` explicitly.
- `run_as` and permissions must be stated, so that ownership does not depend on who ran the command.
- Pipelines must not be marked as development.
- `--cluster-id` is refused outright.
- The current Git branch is checked against the one the target declares, unless `--force` is passed.

One file, two targets

The same `sample_job` resource, resolved for each target

	target dev	target prod
job name	<code>[dev santitham_pro] sample_job</code>	<code>sample_job</code>
trigger	<code>pause_status: PAUSED</code>	<code>pause_status: UNPAUSED</code>
tags	<code>dev: santitham_pro</code>	<code>none</code>
max concurrent runs	<code>4</code>	<code>declared value only</code>
root path	<code>.../.bundle/m3demo/dev</code>	<code>.../.bundle/m3demo/prod</code>

Nothing in the resource file differs between the two columns. The difference is the target, and the mode it declares.

How the configuration is assembled

Five stages, in this order, every time a bundle command runs

- **1 · Read**
databricks.yml is parsed.
- **2 · Merge**
Every file named by include: is merged into the same configuration.
- **3 · Overlay**
The selected target's settings are applied over the common configuration.
- **4 · Preset**
The presets implied by the target's mode are computed and applied.
- **5 · Substitute**
References are replaced by values, except resource ids.

Challenge 1 — Predict the resolved configuration

- 1 A fifteen-line bundle is on the handout. Target dev is selected, and it declares mode: development.
- 2 Without running anything, write down: the resolved job name, the resolved value of the `${var.catalog}` reference, the trigger pause status, and the workspace root path.
- 3 For each answer, write the stage of the assembly sequence that produced it.
- 4 Keep the paper. You check these answers against the CLI in Challenge 2.

From template to running job

Part 2

bundle init and the built-in templates

databricks bundle init <template>, or a directory path, or a Git repository URL

Template	Generates	Requires
default-python	A job with notebook, wheel and pipeline tasks	uv, and a writable catalog
default-minimal	The same databricks.yml skeleton, no resources	nothing further
default-sql	A job running .sql files on a SQL warehouse	a SQL warehouse
default-scala	A JAR job	a JDK and sbt
dbt-sql	A dbt project driven by a job	dbt
pydabs	default-python, with resources declared in Python	uv
mlops-stacks	A full MLOps project with several environments	considerable setup

An organisation distributes its own template as a Git repository, which is how a team standardises the shape of every new project.

Initialisation requires an authenticated workspace

bundle init is not an offline command

```
$ databricks bundle init default-python
```

```
santitham@data-engineer:~/m2-demo/assets$ databricks bundle init default-python  
Welcome to the default Python template for Declarative Automation Bundles!
```

```
Answer the following questions to customize your project.  
You can always change your configuration in the databricks.yml file later.
```

```
Note that https://adb-1502583690645883.3.azuredatabricks.net is used for initialization.  
(For information on how to change your profile, see https://docs.databricks.com/dev-tools/cli/profiles.html.)
```

```
Unique name for this project [my_project]: m3_bundle  
Include a job that runs a notebook: yes  
Include an ETL pipeline: no  
Include a sample Python package that builds into a wheel file: yes  
Use serverless compute: no  
Default catalog for any tables created by this project [ctl_training_dev]:  
Use a personal schema for each user working on this project  
(this is recommended, your personal schema will be 'ctl_training_dev.santitham_pro'): no, I will customize the schema configuration later in databricks.yml
```

```
🌟 Your new project has been created in the 'm3_bundle' directory!
```

```
To get started, refer to the project README.md file and the documentation at https://docs.databricks.com/dev-tools/bundles/index.html.
```

What default-python generates

One command, thirty files. The tree is the reference for Challenge 2.

```
$ databricks bundle init default-python
```

```
$ tree m3_test
```

databricks.yml, pyproject.toml, .gitignore, resources/ with a job and a pipeline, src/ with a package and a notebook, tests/, fixtures/, and editor configuration.

```
santitham@data-engineer:~/m2-demo/assets$ tree m3_bundle/
```

```
m3_bundle/
├── AGENTS.md
├── CLAUDE.md
├── README.md
├── databricks.yml
├── fixtures
├── pyproject.toml
├── resources
│   └── sample_job.job.yml
├── src
│   ├── m3_bundle
│   │   ├── __init__.py
│   │   ├── main.py
│   │   └── taxi.py
│   └── sample_notebook.ipynb
└── tests
    ├── conftest.py
    └── sample_taxi_test.py
```

5 directories, 12 files

The generated databricks.yml

Every construct on this slide was introduced

```
bundle:  
  name: m3_bundle  
  uuid: 4448a889-1924-47fc-bada-83960a085c2d  
  
include:  
  - resources/*.yml  
  
artifacts:  
  python_artifact:  
    type: whl  
    build: uv build --wheel  
  
variables:  
  catalog: {description: The catalog to use}  
  schema:  {description: The schema to use}  
  
targets:  
  dev: {mode: development, default: true, ...}  
  prod: {mode: production, ...}
```

The generated job resource

resources/sample_job.job.yml — three tasks, read rather than written

```
resources:
  jobs:
    sample_job:
      name: sample_job
      trigger:
        periodic: {interval: 1, unit: DAYS}
      parameters:
        - {name: catalog, default: '${var.catalog}'}
        - {name: schema, default: '${var.schema}'}
      tasks:
        - task_key: notebook_task
          notebook_task: {notebook_path: ../src/sample_notebook.ipynb}
        - task_key: python_wheel_task
          depends_on: [{task_key: notebook_task}]
          python_wheel_task: {package_name: m3demo, entry_point: main}
        - task_key: refresh_pipeline
          depends_on: [{task_key: notebook_task}]
          pipeline_task: {pipeline_id: '${resources.pipelines.m3demo_etl.id}'}
```

The remaining generated files

What each is for, and whether it affects the deployment

File or directory	Purpose	Affects deploy
<code>pyproject.toml</code>	Package metadata, the main entry point, dev dependencies	yes
<code>.gitignore</code>	Excludes <code>.databricks/</code> , <code>build/</code> , <code>dist/</code> and explorations	no
<code>src/</code>	The package, the notebook, and the pipeline transformations	yes
<code>tests/</code>	pytest tests and a <code>confest.py</code> , used in Module 5	no
<code>fixtures/</code>	Sample data referenced by the tests	no
<code>.vscode/</code>	Editor settings and recommended extensions	no
<code>AGENTS.md</code> , <code>CLAUDE.md</code>	Instructions for coding agents	no

Challenge 2 — Initialise and inventory

- 1 Run `databricks bundle init default-python`. Give the project the name your instructor supplies, and accept the default catalog it proposes.
- 2 Produce the file tree and compare it against `assets/expected-bundle-tree.txt`. Account for any difference.
- 3 Locate in the generated files: the bundle name, the include pattern, one variable declaration, and the notebook task.
- 4 Run `databricks bundle validate -o json` and check your four Challenge 1 answers.

Success: a tree that matches the reference, and each Challenge 1 answer confirmed or corrected.

bundle validate

Three checks in one command, and the third needs the network

- The assembled configuration matches the bundle schema: keys exist, and values have the type the schema declares.
- Every reference resolves. An undeclared variable or a misspelled namespace fails here.
- The workspace named by the target is reachable, and the credential is accepted.
- This is level 3 of the four levels of verification from Module 2, applied to a second tool. Level 4 is still deployment.

Validation says nothing about whether the workspace will accept the operation. That is the distinction the failure-mode table returns to.

The resolved configuration

bundle validate -o json prints the configuration after all five assembly stages

YOU RUN

```
$ databricks bundle validate -o json \  
  | jq '{presets, variables}'
```

as written in databricks.yml:

```
variables:  
  catalog:  
    description: The catalog to use
```

```
targets:  
  dev:  
    mode: development  
    variables:  
      catalog: training
```

presets appears in no file at all

YOU SEE

```
{  
  "presets": {  
    "jobs_max_concurrent_runs": 4,  
    "name_prefix": "[dev santitham_pro] ",  
    "pipelines_development": true,  
    "tags": { "dev": "santitham_pro" },  
    "trigger_pause_status": "PAUSED"  
  },  
  "variables": {  
    "catalog": {  
      "default": "training",  
      "description": "The catalog to use",  
      "value": "training"  
    }  
  }  
}
```


Local paths become workspace paths

Path rewriting happens during resolution, before anything is deployed

YOU RUN

```
$ databricks bundle validate -o json \  
  | jq '.resources.jobs.sample_job  
      .tasks[0]'
```

as written in the resource file:

```
tasks:  
  - task_key: notebook_task  
    notebook_task:  
      notebook_path:  
        ../src/sample_notebook.ipynb
```

YOU SEE

```
{  
  "task_key": "notebook_task",  
  "notebook_task": {  
    "notebook_path":  
      "/Workspace/Users/santitham.pro@  
      kmutt.ac.th/.bundle/m3demo/dev/  
      files/src/sample_notebook"  
  }  
}
```

the deployed copy, named before it
exists

Challenge 3 — Author a second job

- 1 Write `resources/report.job.yml` declaring one job. Its name must come from a new variable you declare, and it must have exactly one notebook task and no trigger.
- 2 Create the notebook the task names, under `src/`. Two lines are enough.
- 3 Assign the new variable a value in the dev target only.
- 4 Run `databricks bundle validate`, then read the resolved job name in `-o json`.

Success: Validation OK, and a resolved job name carrying the development prefix.

bundle plan

What a deployment would do, reported without doing it

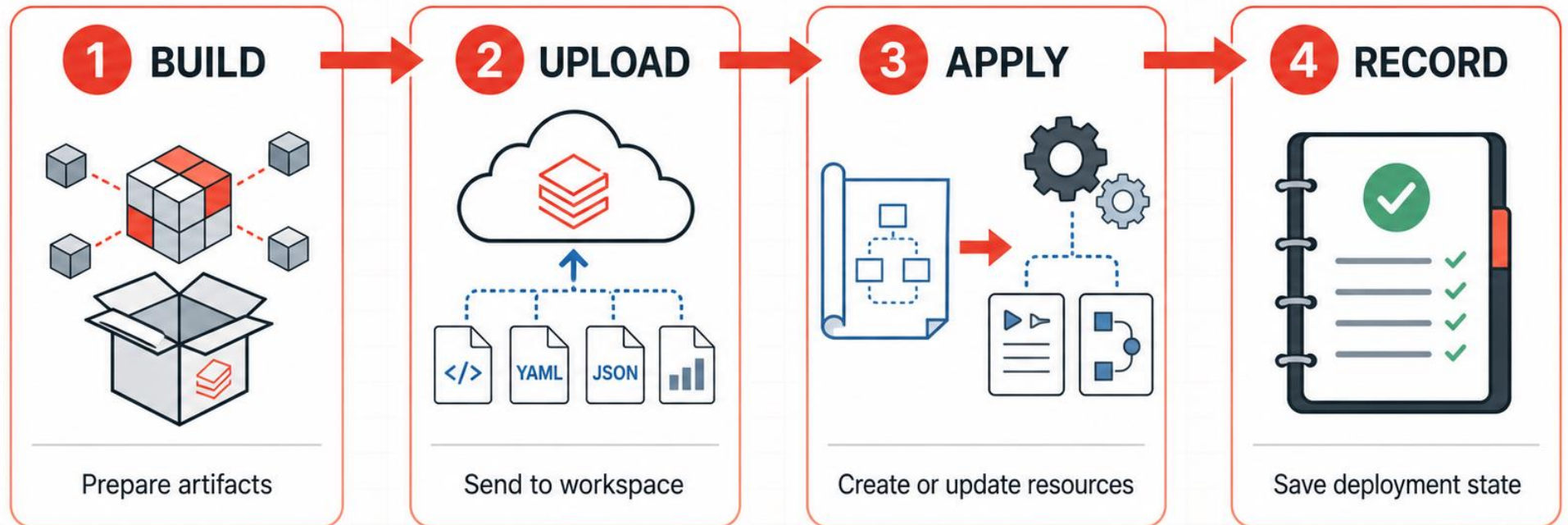
- One line per resource: create, update, delete, or no change.
- It compares the resolved configuration against the deployment state, so the first plan reports every resource as a creation.
- --select restricts the plan, and the same flag restricts a deployment.
- This is the reviewability property from Part A, available on your own machine before it is available in a pull request.

```
santitham@data-engineer:~/m2-demo/assets/m3_bundle$ databricks bundle plan
Building python_artifact...
create jobs.sample_job

Plan: 1 to add, 0 to change, 0 to delete, 0 unchanged
```

Databricks CLI — Bundle Deploy

```
>_ databricks bundle deploy
```



Where the deployment lives

The workspace after a first deployment to target dev

```
santitham@data-engineer:~/m3-demo/assets/m3demo_sp$ databricks bundle validate -t dev
Name: m3demo_sp
Target: dev
Workspace:
  Host: https://adb-1502583690645883.3.azuredatabricks.net
  User: santitham.pro@kmutt.ac.th
  Path: /Workspace/Users/santitham.pro@kmutt.ac.th/.bundle/m3demo_sp/dev

Validation OK!
santitham@data-engineer:~/m3-demo/assets/m3demo_sp$ databricks bundle deploy -t dev
Uploading bundle files to /Workspace/Users/santitham.pro@kmutt.ac.th/.bundle/m3demo_sp/dev/files...
Deploying resources...
Deployment complete!
```

```
santitham@data-engineer:~/m3-demo/assets/m3demo_sp$ databricks workspace list /Users/santitham.pro@kmutt.ac.th/.bundle
```

ID	Type	Language	Path
2018401628068897	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5
3885045965901221	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/orders_pipeline
952328401685898	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_boy
952328401685976	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/OwensCTK
952328401686094	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_praserth
952328401685894	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_heng
952328401686004	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_anurak
952328401686229	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_earth_copy_new
952328401686051	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/kritkaew_m5
952328401685945	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_new
952328401686088	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_sv
952328401686091	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_kp
952328401685979	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_mo
952328401686173	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_layer
952328401686255	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_siriwan
952328401686205	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/cert_m5_pp_test
1913170834193162	DIRECTORY		/Users/santitham.pro@kmutt.ac.th/.bundle/m3demo_sp

Deployment state

How the CLI knows what it already created

- State records the resources this bundle created, so the next deployment computes a difference rather than recreating everything.
- It is held locally under `.databricks/bundle/<target>/` and in the workspace under `state/`. The local directory is git-ignored by the template.
- Bundles created with CLI v1.3.0 or later use the direct engine and write `resources.json`. Earlier bundles use Terraform and write `terraform.tfstate`.
- A bundle carrying both files is refused, reporting the same serial number in terraform and direct states. The migration command is `bundle deployment migrate`.

Challenge 4 — Deploy and inspect

- 1 Run `databricks bundle plan -t dev` and read what it intends to do. Then deploy to dev.
- 2 In the workspace, find the job. Confirm the name prefix and the paused trigger against your Challenge 1 prediction.
- 3 Navigate to the bundle root path and list its four directories.
- 4 State the full workspace path of the notebook this job will execute, and where the same notebook exists in your project directory.

bundle run

One command replaces the trigger, the polling loop and the output retrieval

```
$ databricks bundle run sample_job -t dev
```

```
santitham@data-engineer:~/m3-demo/m3demo$ databricks bundle run sample_job -t dev  
Run URL: https://adb-1502583690645883.3.azuredatabricks.net/jobs/480762596509504/runs/949921444267072?o=1502583690645883  
  
2026-08-16 11:30:13 "[dev santitham_pro] m3_hello" RUNNING  
  
2026-08-16 11:30:37 "[dev santitham_pro] m3_hello" TERMINATED SUCCESS
```


bundle summary and bundle open

Two commands that read deployment state rather than configuration

SUMMARY

```
$ databricks bundle summary -t dev
```

Lists every resource this bundle has deployed to the selected target, with the workspace id of each.

Empty before the first deployment, because state is what it reads.

OPEN

```
$ databricks bundle open sample_job
```

Opens the resource in a browser, resolving the id from state.

Useful when a run has failed and the next step is the run page rather than the terminal.

Challenge 5 — Run the job and read the result

- 1 Run the job from the command line and record the run URL it prints.
- 2 Report the resources this bundle has deployed, and the workspace id of the job.
- 3 Open the run and confirm which notebook path the task executed.
- 4 Compare that path with the one you wrote down in Challenge 4.

bundle summary

```
$ databricks bundle summary -t dev
```

```
santitham@data-engineer:~/m3-demo/m3demo$ databricks bundle summary -t dev
```

```
Name: m3demo
```

```
Target: dev
```

```
Workspace:
```

```
  User: santitham.pro@kmutt.ac.th
```

```
  Path: /Workspace/Users/santitham.pro@kmutt.ac.th/.bundle/m3demo/dev
```

```
Resources:
```

```
  Jobs:
```

```
    sample_job:
```

```
      Name: [dev santitham_pro] m3_hello
```

```
      URL: https://adb-1502583690645883.3.azuredatabricks.net/jobs/480762596509504
```

bundle destroy

Deletes the resources this bundle created, then the files it uploaded

```
$ databricks bundle destroy -t dev
```

```
santitham@data-engineer:~/m3-demo/m3demo$ databricks bundle destroy -t dev
```

```
The following resources will be deleted:
```

```
delete resources.jobs.sample_job
```

```
All files and directories at the following location will be deleted: /Workspace/Users/santitham.pro@kmutt.ac.th/.bundle/m3demo/dev
```

```
Would you like to proceed? [y/N]: y
```

```
Deleting files...
```

```
Destroy complete!
```

What destroy does not remove

The boundary is the deployment state

- Tables and files written by a job that ran. Destroying the job does not touch what it produced.
- Catalogs and schemas that the bundle referenced rather than declared.
- Resources created by hand in the workspace, even where they carry the same name.
- Anything deployed under a different target. Destroy acts on one target at a time.

Challenge 6 — Destroy, redeploy, override

- 1 Destroy the dev deployment and confirm from the workspace that the job is gone.
- 2 Redeploy, passing `--var` to override the variable you declared in Challenge 3.
- 3 Confirm the override in the resolved configuration, and then in the workspace.
- 4 Name which of the four sources of a variable value won, and why.

Your code exists twice

After a deployment, the same notebook exists in two places with different roles

THE SOURCE

```
my-project/src/sample_notebook.ipynb
```

What you edit. What the repository holds. What a reviewer reads.

Editing it changes nothing in the workspace until you deploy.

THE DEPLOYED COPY

```
.bundle/m3demo/dev/files/src/  
sample_notebook
```

What the job executes.

Editing it in the workspace changes what runs, and the next deployment overwrites the edit without warning.

The task path, read on the validate slide or from the run page, states which copy a job is running.

Six failure modes and their signals

In the order you are likely to meet them

Symptom	Cause	Detected
<code>cannot configure default credentials</code>	No profile, and no environment variables	before init writes a file
<code>Forbidden on the metastore call</code>	Credential without Unity Catalog access	before init writes a file
<code>uv: command not found</code>	uv absent; the template builds a wheel with it	build stage of deploy
<code>Validation OK, then deploy fails</code>	The schema is satisfied; the workspace refuses the operation	apply stage of deploy
<code>The job runs an old version</code>	The deployed copy was edited, or the source was not deployed	not detected; read the task path
<code>Deploy to prod refused</code>	Branch check, or user-specific paths under production mode	before any change is made

What this module did not cover

Each of these has a module of its own

- **Authoring jobs, tasks and cluster specifications**
Module 4, this afternoon. You read the generated resource file today; tomorrow you write one, with several tasks and a declared cluster.
- **Promotion across more than two targets, and variable overrides in earnest**
Module 4.
- **Diagnosing a deployment that fails in the apply stage**
Module 4, with a deliberately sabotaged bundle.
- **Tests over the code a bundle deploys, and running these commands on a server**
Modules 5 and 7.

The End of Module 3

If you have any further questions,
please contact me via
santitham.pro@kmutt.ac.th

